Blood Bank Management System (BBMS)

1. Introduction

Blood Bank Management System (BBMS) is a software system used to manage blood donation, storage, and distribution efficiently. It maintains donor information, blood stock levels, and handles emergency blood requests.

In traditional systems, blood records are maintained manually, which may lead to errors, delays, and difficulty in emergency situations. This system automates the process to ensure **quick access, accuracy, and proper tracking of blood availability**.

The main focus of this system is **tracking blood stock and generating alerts in emergency situations**, ensuring timely availability of blood.

2. Objectives

- To automate blood bank operations
- To maintain donor information efficiently
- To track blood stock and availability
- To manage blood donation records
- To handle blood requests quickly
- To generate alerts during emergencies
- To ensure proper inventory control

3. Functional Requirements

The system should be able to:

- Register new donors
- Store donor details (blood group, last donation date)
- Record blood donations
- Maintain blood stock information
- Track blood quantity by group
- Accept blood requests from patients
- Match required blood group with available stock
- Issue blood to recipients
- Generate alerts for low stock or emergency
- Maintain donation and request history

4. Non-Functional Requirements

The system should:

- Provide fast response in emergency situations
- Ensure data accuracy and integrity
- Be secure and protect donor information
- Be user-friendly and easy to operate
- Support backup and recovery
- Be reliable and scalable

5. Use Case Modeling

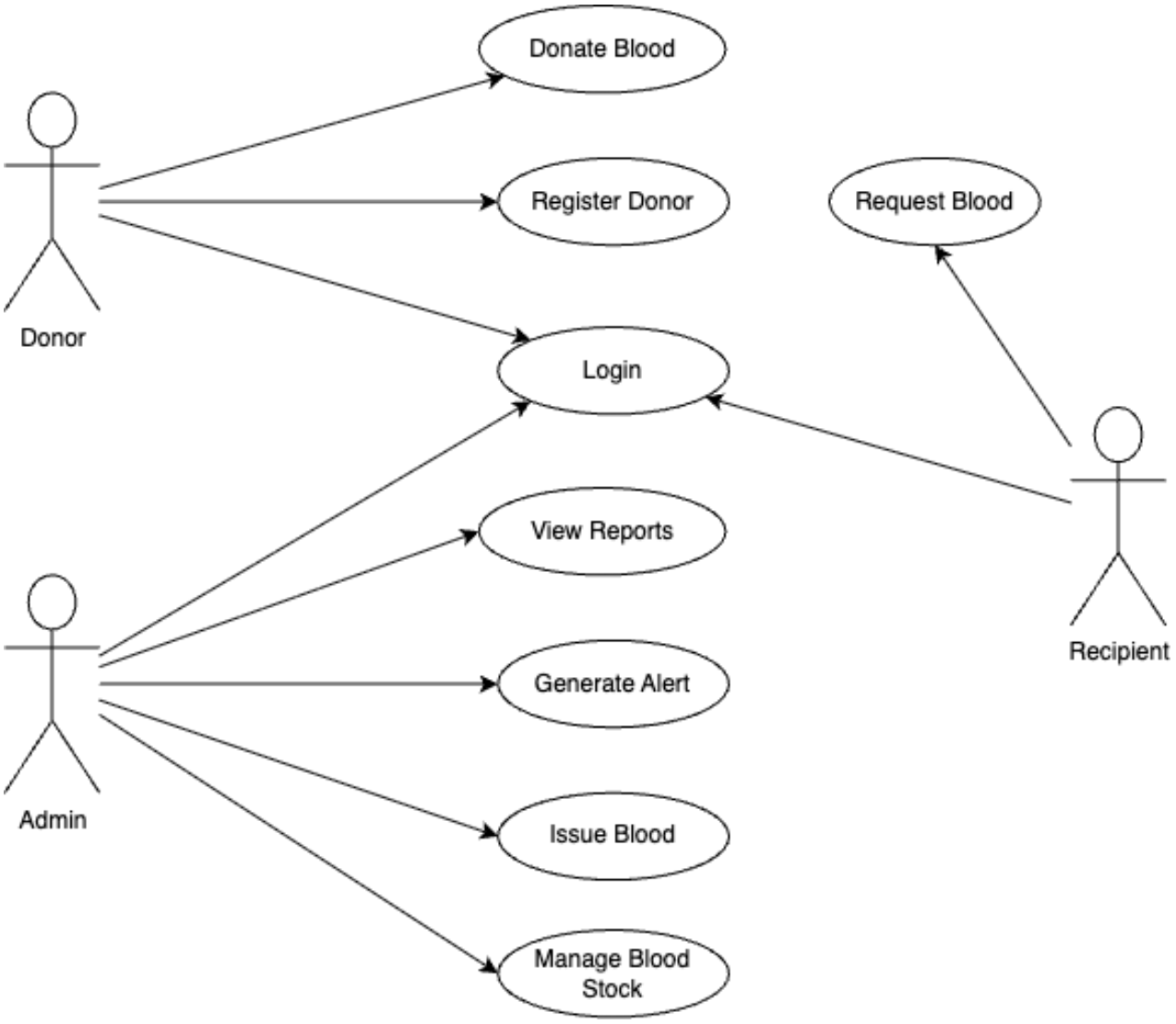
Actors:

- Donor
- Recipient
- Admin

Main Use Cases:

- Register Donor
- Login
- Donate Blood
- Manage Blood Stock
- Request Blood
- Issue Blood
- Generate Alert
- View Reports

Use Case Diagram:



6. Structural Design (Class Diagram)

Main Classes:

- User
- Donor
- Recipient
- BloodStock
- Donation
- BloodRequest
- Alert

Description:

- User is the parent class
- Donor and Recipient inherit from User
- Donor donates blood
- BloodStock stores available blood units
- Donation records donation details
- BloodRequest handles blood requests
- Alert notifies low stock or emergency

Attributes and Methods

User

- Attributes: id, name
- Method: login()

Donor

- Attributes: blood_group, last_donation_date
- Method: donateBlood()

Recipient

- Method: requestBlood()

BloodStock

- Attributes: blood_group, quantity

Donation

- Attributes: donation_date, quantity

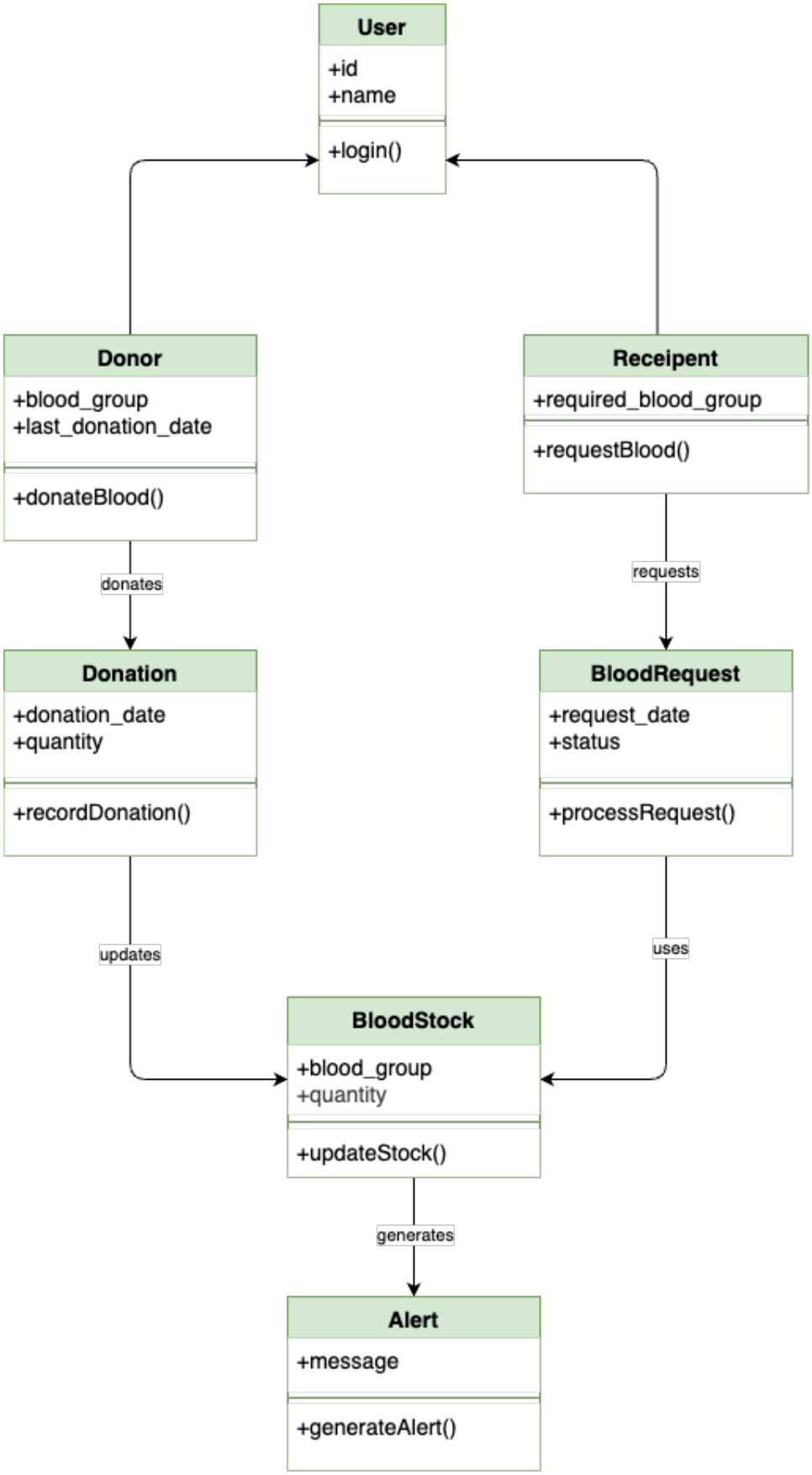
BloodRequest

- Attributes: request_date, status

Alert

- Attribute: message

Class Diagram



7. Database Design (ER Diagram)

7.1 Entities

- Donor
- Blood_Stock
- Recipient
- Donation
- Blood_Request
- Alert

7.2 Relationships

- Donor donates Blood (Donation)
- Donation updates Blood Stock
- Recipient requests Blood
- Blood Request uses Blood Stock
- Blood Stock generates Alert

7.3 Suggested Primary Keys and Foreign Keys

DONOR

- donor_id (PK)
- name
- blood_group
- phone
- last_donation_date

BLOOD_STOCK

- stock_id (PK)
- blood_group
- quantity

RECIPIENT

- recipient_id (PK)
- name
- required_blood_group

DONATION

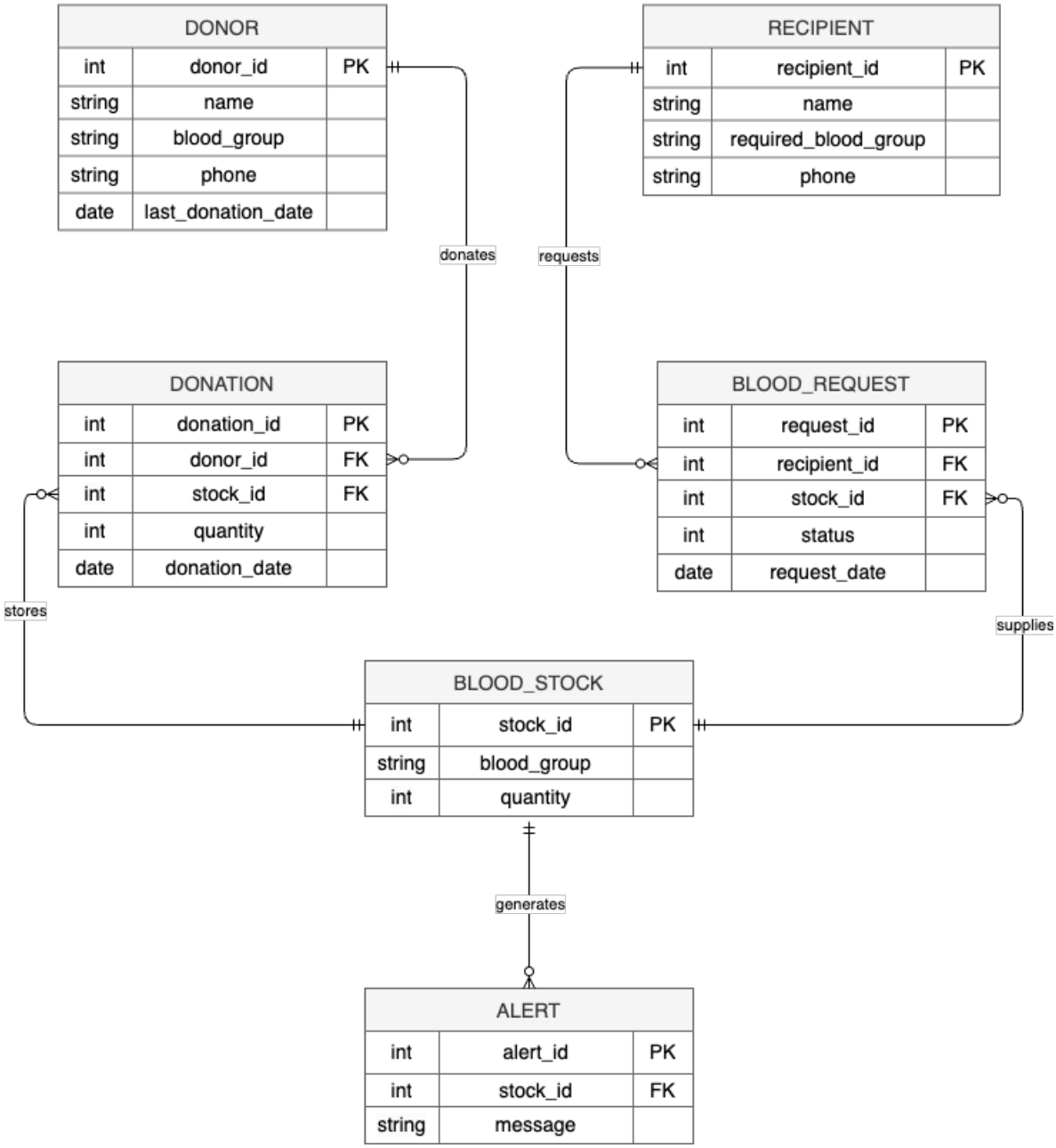
- donation_id (PK)
- donor_id (FK)
- stock_id (FK)
- donation_date
- quantity

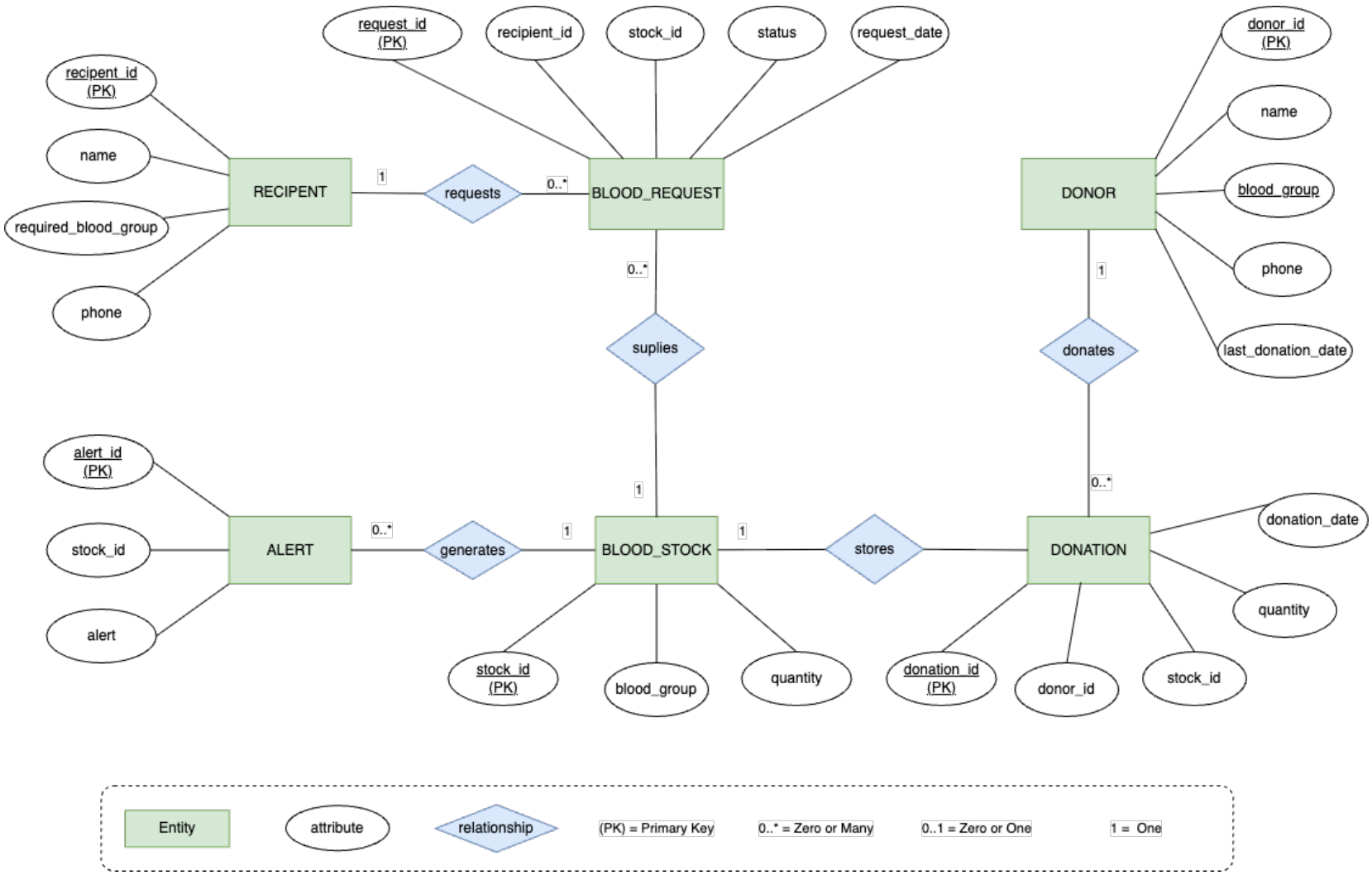
BLOOD_REQUEST

- request_id (PK)
- recipient_id (FK)
- stock_id (FK)
- request_date
- status

ALERT

- alert_id (PK)
- stock_id (FK)
- message




RECIPENT
recipent_id (PK)
name
required_blood_group
phone
requests
1
0..*
BLOOD_REQUEST
request_id (PK)
recipient_id
stock_id
status
request_date
suplies
0..*
1
BLOOD_STOCK
stock_id (PK)
blood_group
quantity
ALERT
alert_id (PK)
stock_id
alert
generates
0..*
1
stores
1
1
DONOR
donor_id (PK)
name
blood_group
phone
last_donation_date
donates
1
0..*
DONATION
donation_id (PK)
donor_id
stock_id
quantity
donation_date
Entity
attribute
relationship
(PK) = Primary Key
0..* = Zero or Many
0..1 = Zero or One
1 = One

8. Advantages

- Ensures quick access to blood in emergencies
- Reduces manual errors
- Improves inventory management
- Maintains accurate records
- Saves time
- Supports alert system for low stock

9. Limitations

- Requires technical knowledge
- Initial setup cost is high
- System failure may affect emergency services
- Needs proper data security

10. Conclusion

The Blood Bank Management System is an essential solution for managing blood donation and distribution efficiently. It ensures proper tracking of blood stock, quick response to emergency requests, and reliable alert generation. The system improves efficiency, reduces errors, and enhances life-saving operations in healthcare environments.

11. Simple Web Implementation

index.html

```html
<!DOCTYPE html>
<html>
<head>
  <title>Blood Bank Management System</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<header>
  <h1>Blood Bank Management System</h1>
  <p>Simple Offline Blood Bank Portal</p>
</header>

<nav>
```

```html
  <a href="#donor">Donor</a>
  <a href="#donate">Donate</a>
  <a href="#request">Request</a>
  <a href="#stock">Stock</a>
  <a href="#history">History</a>
</nav>

<div class="container">

  <section id="donor" class="card">
    <h2>Register Donor</h2>
    <form id="donorForm">
      <input type="text" id="dname" placeholder="Donor Name" required>
      <input type="text" id="dblood" placeholder="Blood Group" required>
      <input type="text" id="dphone" placeholder="Phone Number" required>
      <button type="submit">Register Donor</button>
    </form>
  </section>

  <section id="donate" class="card">
    <h2>Donate Blood</h2>
    <form id="donateForm">
      <input type="text" id="bloodGroup" placeholder="Blood Group" required>
      <input type="number" id="units" placeholder="Units" required>
      <button type="submit">Donate</button>
    </form>
  </section>

  <section id="request" class="card">
    <h2>Request Blood</h2>
    <form id="requestForm">
      <input type="text" id="rname" placeholder="Patient Name" required>
      <input type="text" id="rblood" placeholder="Required Blood Group" required>
      <input type="number" id="runits" placeholder="Units" required>
      <button type="submit">Request Blood</button>
    </form>
  </section>

  <section id="stock" class="card">
    <h2>Blood Stock</h2>
    <div id="stockList"></div>
  </section>

  <section id="history" class="card">
    <div class="section-title">
      <h2>Request History</h2>
      <button onclick="clearRequests()" class="danger-btn">Clear</button>
    </div>
    <div id="requestList"></div>
  </section>
```

```html
</div>

<script src="script.js"></script>
</body>
</html>
```

styles.css

```css
body {
  font-family: Arial;
  background: #f4f7fb;
  margin: 0;
}

header {
  background: #dc2626;
  color: white;
  text-align: center;
  padding: 25px;
}

nav {
  background: white;
  text-align: center;
  padding: 12px;
  box-shadow: 0 3px 10px rgba(0,0,0,0.1);
}

nav a {
  margin: 10px;
  color: #dc2626;
  text-decoration: none;
  font-weight: bold;
}

.container {
  width: 85%;
  margin: auto;
}

.card {
  background: white;
  padding: 20px;
  margin: 20px 0;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0,0,0,0.08);
}

input {
  display: block;
  width: 100%;
  padding: 11px;
```

```
  margin: 10px 0;
  box-sizing: border-box;
}

button {
  background: #dc2626;
  color: white;
  border: none;
  padding: 10px 14px;
  border-radius: 5px;
  cursor: pointer;
}

button:hover {
  background: #1c60b9;
}

.stock-card {
  border-left: 5px solid #dc2626;
}

.low {
  color: red;
  font-weight: bold;
}

.safe {
  color: green;
  font-weight: bold;
}

.issued {
  color: green;
  font-weight: bold;
}

.rejected {
  color: red;
  font-weight: bold;
}

.section-title {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.danger-btn {
  background: #7f1d1d;
}
```

script.js

```
const stockList = document.getElementById("stockList");
const requestList = document.getElementById("requestList");

// Default blood stock
const defaultStock = {
  "A+": 5,
  "A-": 3,
  "B+": 4,
  "B-": 2,
  "O+": 6,
  "O-": 2,
  "AB+": 3,
  "AB-": 1
};

// ---------- STORAGE ----------
function getStock() {
  let stock = localStorage.getItem("bloodStock");

  if (stock) {
    return JSON.parse(stock);
  } else {
    localStorage.setItem("bloodStock", JSON.stringify(defaultStock));
    return defaultStock;
  }
}

function saveStock(stock) {
  localStorage.setItem("bloodStock", JSON.stringify(stock));
}

function getRequests() {
  return JSON.parse(localStorage.getItem("bloodRequests")) || [];
}

function saveRequests(requests) {
  localStorage.setItem("bloodRequests", JSON.stringify(requests));
}

function getDonors() {
  return JSON.parse(localStorage.getItem("donors")) || [];
}

function saveDonors(donors) {
  localStorage.setItem("donors", JSON.stringify(donors));
}

// ---------- REGISTER DONOR ----------
document.getElementById("donorForm").addEventListener("submit", function(e) {
```

```
  e.preventDefault();

  const name = document.getElementById("dname").value;
  const blood = document.getElementById("dblood").value.toUpperCase();
  const phone = document.getElementById("dphone").value;

  const donors = getDonors();

  donors.push({
    name: name,
    blood: blood,
    phone: phone,
    date: new Date().toLocaleDateString()
  });

  saveDonors(donors);

  alert("Donor registered successfully.");
  e.target.reset();
});

// ---------- DONATE BLOOD ----------
document.getElementById("donateForm").addEventListener("submit", function(e) {
  e.preventDefault();

  const group = document.getElementById("bloodGroup").value.toUpperCase();
  const units = parseInt(document.getElementById("units").value);

  const stock = getStock();

  stock[group] = (stock[group] || 0) + units;

  saveStock(stock);
  renderStock();

  alert("Blood added to stock.");
  e.target.reset();
});

// ---------- REQUEST BLOOD ----------
document.getElementById("requestForm").addEventListener("submit", function(e) {
  e.preventDefault();

  const patient = document.getElementById("rname").value;
  const group = document.getElementById("rblood").value.toUpperCase();
  const units = parseInt(document.getElementById("runits").value);

  const stock = getStock();
  const requests = getRequests();

  let status = "Rejected";
```

```
  if (stock[group] && stock[group] >= units) {
    stock[group] = stock[group] - units;
    status = "Issued";
    saveStock(stock);
  }

  requests.push({
    patient: patient,
    group: group,
    units: units,
    date: new Date().toLocaleDateString(),
    status: status
  });

  saveRequests(requests);
  renderStock();
  renderRequests();

  if (status === "Issued") {
    alert("Blood issued successfully.");
  } else {
    alert("Not enough blood. Request rejected.");
  }

  e.target.reset();
});

// ---------- SHOW STOCK ----------
function renderStock() {
  const stock = getStock();

  stockList.innerHTML = "";

  for (let group in stock) {
    let qty = stock[group];

    let statusText = "Available";
    let statusClass = "safe";

    if (qty <= 2) {
      statusText = "Low Stock Alert";
      statusClass = "low";
    }

    stockList.innerHTML += `
      <div class="card stock-card">
        <p><b>Blood Group:</b> ${group}</p>
        <p><b>Quantity:</b> ${qty} units</p>
        <p><b>Status:</b> <span class="${statusClass}">${statusText}</span></p>
      </div>
```

```
    `;
  }
}

// ---------- SHOW REQUEST HISTORY ----------
function renderRequests() {
  const requests = getRequests();

  if (requests.length === 0) {
    requestList.innerHTML = "<p>No request history available.</p>";
    return;
  }
  requestList.innerHTML = "";

  requests.forEach(function(req) {
    let cls = req.status === "Issued" ? "issued" : "rejected";

    requestList.innerHTML += `
      <div class="card">
        <p><b>Patient:</b> ${req.patient}</p>
        <p><b>Blood Group:</b> ${req.group}</p>
        <p><b>Units:</b> ${req.units}</p>
        <p><b>Date:</b> ${req.date}</p>
        <p><b>Status:</b> <span class="${cls}">${req.status}</span></p>
      </div>
    `;
  });
}

// ---------- CLEAR REQUEST HISTORY ----------
function clearRequests() {
  localStorage.removeItem("bloodRequests");
  renderRequests();
  alert("Request history cleared.");
}

function clearAllData() {
  localStorage.setItem("bloodStock", JSON.stringify(defaultStock));
  localStorage.removeItem("bloodRequests");
  localStorage.removeItem("donors");

  renderStock();
  renderRequests();

  alert("All data reset successfully.");
}
// ---------- INITIAL LOAD ----------
renderStock();
renderRequests();
```